Advances in Intelligent Transportation Systems in Smart Cities - Research Article

International Journal of Distributed Sensor Networks

International Journal of Distributed Sensor Networks
2019, Vol. 15(12)

DOI: 10.1177/1550147719895956
journals.sagepub.com/home/dsn
SAGE

Reservation-based traffic management for autonomous intersection crossing

Myungwhan Choi[1], Areeya Rubenecia[1] and Hyo Hyun Choi[2]

Abstract

A scheduling scheme for autonomous intersection crossing is proposed and evaluated. The scheduling scheme determines the flow of autonomous vehicles into an intersection without traffic signals. The objective of the scheduling scheme is to schedule the vehicles' entrances into the intersection such that there will be no collision among vehicles and the intersection is efficiently utilized. Our scheduling scheme uses reservation-based scheduling approach, and the scheduling is formulated as an optimization problem to find the best sequence of vehicles' entrances into the intersection. Our scheme is made more practical by allowing the vehicles to move at any speed within a speed range, and it is shown that it is fast enough to be used in real time. Through simulation, it is also shown that our scheduling scheme significantly outperforms the first-come, first-served approach, which is the approach used by other reservation-based scheduling schemes.

Keywords

Autonomous driving, intersection traffic management, optimization, reservation-based scheduling

Date received: 30 June 2019; accepted: 4 November 2019

Handling Editor: Rodolfo Meneguette

Introduction

Autonomous vehicle (AV) is one of the key components of the Intelligent Transport System (ITS). Compared with human drivers, AVs can reduce crashes since they can more accurately monitor and quickly react to its surroundings. They are also expected to follow traffic rules which will eliminate the problems common with human driving such as over-speeding and drunk driving. In addition, AVs can also improve traffic flows, reduce energy or fuel consumption, and reduce vehicle emissions. Hence, many vehicle manufacturers have been showing huge investment in the research and development in this field.[1]

ITS works by equipping the AVs and infrastructure with sensors, cameras, and so on, to collect useful data such as the position, speed, and destination of a vehicle. The collected data are then exchanged among the vehicles and the road-side equipment through vehicle-to-vehicle (V2V) or through vehicle-to-infrastructure (V2I) communications. This obtained information is analyzed and used to plan the future actions of the vehicles.

In the conventional traffic system, traffic signals are used to control the flow of the vehicles in the intersection. A problem with the use of traffic signals is that they do not efficiently utilize the intersection especially when the traffic is low. For example, vehicles sometimes have to wait in the waiting area before they can enter the intersection because the traffic signal is red even though no other vehicles are in the intersection. This

[1]Department of Computer Science and Engineering, Sogang University, Seoul, Republic of Korea
[2]Department of Computer Science, Inha Technical College, Incheon, Republic of Korea

Corresponding author:
Hyo Hyun Choi, Department of Computer Science, Inha Technical College, Incheon 22212, Republic of Korea.
Email: hchoi@inhatc.ac.kr

problem can be solved by introducing the intersection traffic management system, which does not use traffic signals. A study that introduced autonomous intersection management (AIM)[2] has presented an intersection control mechanism that coordinates the vehicles' routes across the intersection based on the actual observed traffic. Results in Dresner and Stone's paper[2] show that AIM improves the efficiency of the intersection compared to the intersection that uses traffic signals.

Aside from improving the intersection's efficiency, it is also important to ensure the safety of the vehicles in the intersection, which means vehicles must be able to travel across the intersection without collision. Crashes are likely to happen in the intersection since vehicles from different directions converge to one common space.[3] Consequently, most studies in AIM have focused on the safety issue and the efficiency improvement in the intersection.

The main objective of this study is to find a collision-free and highly efficient scheme to manage the travel of AVs in the intersection. Our scheduling scheme uses reservation-based scheduling approach. We schedule multiple vehicles in every scheduling phase and use genetic algorithm to find the sequence of vehicles' entrances to the intersection that minimizes the overall delay of vehicles while also considering the fairness of the scheduling. This is different from other reservation-based scheduling schemes since they take a first-come, first-served (FCFS) approach wherein the vehicles are scheduled in the order that their requests are received by the intersection manager. Moreover, we relax the common requirement that vehicles must always move at a constant speed in the intersection. In our proposed scheme, vehicles can pass the intersection at any speed in the predefined speed range, which makes the proposed scheme more practical.

The rest of the article is organized as follows. In the "Related work" section, we discuss related studies to the AIM. In the "System overview" section, we give a brief background on the reservation-based scheduling, and also describe the intersection environment and the expected behaviors of the vehicles in our system. In the "Scheduling scheme" and "Vehicles can travel in intersection at any speed within the allowed speed range" sections, the proposed scheduling scheme is described in detail. The performance of our scheme is shown in the "Performance evaluation" section, and finally the "Conclusion" section concludes the article.

Related work

Dresner and Stone[2] introduced the reservation-based approach to ensure safety in the intersection, and it is extended by de La Fortelle.[4] It is assumed in de La Fortelle's paper[4] that the vehicles move in the intersection at the predetermined maximum speed. By making all the vehicles in the intersection move at the given highest speed, the highly efficient usage of the intersection is expected. In Azimi et al.'s paper,[5] the intersection is divided into cells and only one vehicle is allowed to occupy a cell at a time. Differently from other works, however, in Azimi et al.'s paper,[5] the vehicles communicate with each other to coordinate their routes without using the V2I communications platform. In the case when there is a conflict between the routes, FCFS policy is used to decide which vehicle can enter the intersection first.

Other methods proposed in the previous studies[6–9] do not discretize the intersection. In Yan et al.'s study,[6] they identified the compatible trajectories of the vehicles in the intersection which are the trajectories that are not in conflict with each other. Then, they group the vehicles with compatible trajectories together and only allow vehicles within the same group to travel across the intersection at the same time. Similar approach with Yan et al.[6] is taken in Li and Wang[7] wherein they only allow vehicles with non-conflicting trajectories to travel the intersection at the same time. Van Middlesworth et al.[8] proposed a scheme for small and low-traffic intersections, which use V2V communication. The vehicles communicate with each other to broadcast their upcoming entrances into the intersection, and the scheme has set of rules to be followed by the vehicles in case the vehicles have conflicting trajectories in the intersection. In Lu and Kim's paper,[9] instead of discretizing the intersection, the vehicle's actual occupancy is used. The motivation of Lu and Kim[9] is to overcome the increase in computational complexity of the reservation-based scheduling when the granularity of cells is high. Hence, instead of discretizing the intersection, the discrete time sequence of the vehicles' actual occupancy in the intersection is obtained and used to detect conflicting trajectories.

In our work, we use the reservation-based approach as proposed by others to solve the issue of collision avoidance in the intersection. Differently from other approaches, however, we do not use the FCFS approach, and we also allow the vehicles to move at any speed within the predetermined speed range to make the proposed scheme more practical.

A study in Levin et al.[10] shows that the system with traffic signals can perform better than the reservation-based scheduling approach on realistic examples. In Levin et al.,[10] the FCFS reservations system is compared with traffic signal system on downtown Austin subnetwork. Using FCFS scheme, vehicles are scheduled in the order their requests are received by the intersection manager. Therefore, FCFS scheme will show better performance than traffic signal system when the arrival rate of the vehicles is low. But when traffic is heavy, the traffic signal system will show better

performance because it has a property that allows vehicles to enter the intersection by batch. Thus, when traffic is high, we need to devise a more sophisticated scheduling scheme than FCFS, and this is a part of our main work.

Dynamic programming can be used to find optimal sequence of vehicles' entrances into the intersection as used in Wu et al.'s[11] and Yan et al.'s[6] studies, but the complexity is not acceptable for real-time systems. Li and Wang[7] enumerated all possible sequences and chose the most efficient sequence. However, the complexity also increases as the number of vehicles increases. Some solutions used to find near-optimal solutions include ant colony algorithm and genetic algorithm. A heuristic based on an ant colony system is proposed in Wu et al.[12] Genetic algorithm is used in Lin et al.,[13] while active set method and interior point method are used along with genetic algorithm in Lee and Park.[14]

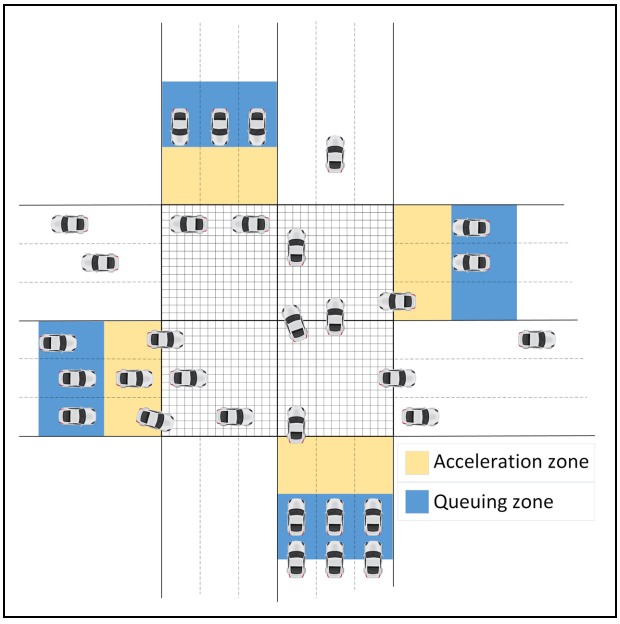

Figure 1. The intersection model used in our scheduling scheme.

System overview

Background on reservation-based scheduling

Our scheduling scheme uses the reservation-based scheduling introduced in Dresner and Stone's study.[2] In a reservation-based scheduling, the intersection is divided into multiple sections, which we call cells. When a vehicle travels along the intersection, it will occupy a sequence of cells. For the efficient use of the intersection, multiple vehicles should be allowed to be in the intersection at the same time. To implement this approach, a cell needs to be reserved to only one vehicle for a certain period of time. More information on how the scheduling is done will be discussed in the future section.

Intersection model

We consider a four-way intersection with three lanes per way as shown in Figure 1. The intersection is divided into multiple cells, and the road consists of queuing zone and acceleration zone. The queuing zone is where the vehicles can send their reservation requests, while the acceleration zone is where vehicles adjust their speeds so that they enter the intersection as scheduled.

The two main parts in our model are the intersection manager and the vehicle. Their roles and expected actions are discussed as follows:

Intersection manager

- It receives the reservation requests from the vehicles, schedules their entrance into the intersection, and sends responses to them. Details on the scheduling will be discussed in the later section.
- It processes requests every discrete time instant t_i, where $t_i - t_{i-1} = \Delta t$, time step interval. At t_i, the intersection manager processes the requests received during the interval $[t_{i-1}, t_i)$, and the processing should be completed within Δt.

Vehicle

- It is autonomous and is supposed to follow the instructions given by the intersection manager. Human driven vehicles are not allowed.
- Aside from ordinary vehicle, it can be an emergency or a transit vehicle. Preferential treatment is given to emergency vehicles and transit vehicles in scheduling.
- It generates its own reservation request message before it enters the queuing zone. How to generate the reservation request message will be discussed in the later section.
- It can send its reservation request message to the intersection manager only when it is in the queuing zone and it does not have a reservation yet. After sending the request message, it will receive either an acceptance message or a rejection message. If a rejection message is received, it sends the reservation request message again. This will be repeated until it receives an acceptance message. An acceptance message means that it has been successfully processed, and the vehicle must enter the entrance of the acceleration zone as scheduled. The acceptance message includes the time when the vehicle should enter the acceleration zone. More information about the rejection

and acceptance message will be discussed in the later section.

- Once it enters the queuing zone through a lane, it is to stay at that lane and is not allowed to overtake any vehicles ahead.
- In the queuing zone, it can travel at any speed not exceeding the maximum speed in the lane $s_{\max,\text{lane}}$, when there is no concern over collision. Otherwise, it will slow down and come to a stop to avoid collision with a vehicle ahead. The value of $s_{\max,\text{lane}}$ must be less than or equal to $s_{\max,\text{inter}}$ which is the maximum allowed speed in the intersection.
- It cannot enter the acceleration zone without a reservation and can only enter the acceleration zone as scheduled.
- Once it enters the acceleration zone, it will either accelerate or maintain its speed so that its speed when it enters the intersection is within the allowed speed range $[s_{\min,\text{inter}}, s_{\max,\text{inter}}]$.
- In the intersection, it can turn left, turn right, or go straight. Its speed must be within the allowed speed range while traveling in the intersection.
- It can be of any size and is assumed to be rectangular in shape.
- We assume that there is no message transmission delay.

Length of the queuing zone. Since a vehicle can only send request when it is in the queuing zone and it cannot enter the acceleration zone without reservation, we want to avoid the situation where a vehicle will have to stop at the entrance of the acceleration zone simply because it does not have the reservation by the time it reaches there. To find a way to avoid this situation, it is important to notice that up to one Δt is spent before the scheduling process starts for the received intersection message by the intersection manager and then additional Δt is required to complete the processing of the request. That is, a vehicle after sending its reservation request message has to wait up to $2\Delta t$ before it gets response for the reservation request. Thus, to make sure that a vehicle traveling at $s_{\max,\text{lane}}$, the fastest allowed speed in the queuing zone, will receive its schedule before it reaches the entrance of the acceleration zone, the length of the queuing zone must be at least $2\Delta t s_{\max,\text{lane}}$. Then, a vehicle traveling at a speed slower than $s_{\max,\text{lane}}$ will be in the queuing zone for longer than $2\Delta t$ and therefore will receive its schedule before reaching the entrance of the acceleration zone.

Size of the cell. The granularity of the cells has an effect to the efficiency of the intersection. A cell will be reserved for the vehicle until the vehicle departs from the cell and another vehicle can reserve the cell only after the previous reservation. Moreover, the intersection space can be better utilized by making the cell size smaller since this enables the vehicles to more accurately determine the intersection area that they will occupy for a given time. Thus, using smaller cell size increases the traffic throughput and the intersection's efficiency. However, decreasing the cell size also increases the scheduling run time. Hence, it is important to use an appropriate cell size that is beneficial to the intersection's performance, and at the same time, the scheduling time is fast enough to be used in real time.

Intersection crossing plans

A vehicle can either move straight, turn left, or turn right while traveling in the intersection. To minimize the conflict among the trajectories of various vehicles that are in the intersection at the same time, we introduce the intersection crossing plans. The intersection crossing plan determines the lanes that a vehicle can take according to its moving direction as shown in Figure 2. This is applied to all of the four ways.

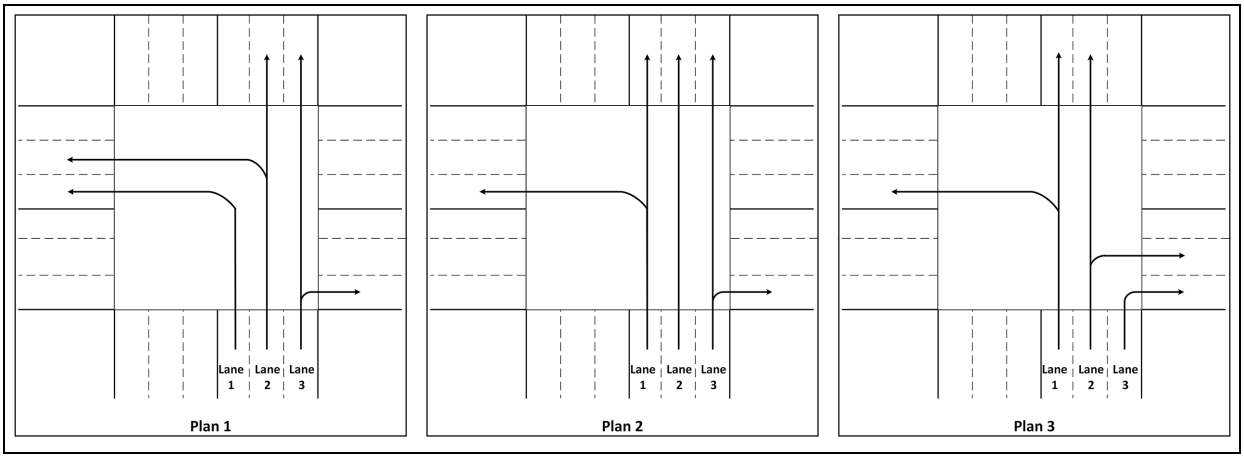

Figure 2. The lanes a vehicle can take based on the crossing plans.

We have three intersection crossing plans that we can apply to the intersection model. In plan 1, vehicles that will turn left can take lane 1 or lane 2, vehicles going straight can take lane 2 or lane 3, and vehicles turning right can take lane 3. In plan 2, vehicles turning left can take lane 1, vehicles going straight can take lane 1 or lane 2 or lane 3, and vehicles turning right can take lane 3. In plan 3, vehicles turning left must take lane 1, vehicles going straight can take lane 1 or lane 2, and vehicles turning right can take lane 2 or lane 3. When a vehicle can go to more than one lane, the vehicle can be assigned to any lane or it can be assigned to the lane with fewer vehicles in queue.

Based on the actual traffic, we can assign an appropriate intersection crossing plan to be used. For example, when majority of the vehicles in the system are turning left, then we can use the intersection crossing plan 1 since in this crossing plan, two lanes are assigned for left-turning vehicles.

Scheduling scheme

Our scheduling scheme consists of two key parts: the generation of the reservation request message and the scheduling of the vehicles based on their reservation requests. The generation of the reservation request will be discussed in the "Vehicle generates reservation request message" section, while the scheduling of the vehicles will be explained in the "Intersection manager schedules the requests" section.

Vehicle generates reservation request message

As previously mentioned, a vehicle must send its reservation request to the intersection manager so that the intersection manager can schedule its entrance to the intersection. In this section, we describe the information contained in the reservation request message and explain how to obtain this information.

Reservation request message. When a vehicle sends a request, its reservation request message contains the following information:

- Vehicle ID;
- Vehicle type: emergency, transit, or ordinary;
- *rc*: list of cells that the vehicle will occupy in the intersection;
- *rta*: list of starting times for the requested reservation time intervals for the cells in *rc*;
- *rtd*: list of ending times for the requested reservation time intervals for the cells in *rc*.

Reservation request generation. To determine the cells that a vehicle will occupy and the reservation times for them, we simulate the vehicle's movement in the intersection.

We must reserve cells considering the requirement that the vehicles can travel at any speed within the speed range. Consider an arbitrary cell in the intersection, $cell_A$, that a vehicle will pass through in the intersection. The earliest time that the vehicle will arrive at $cell_A$ is when the vehicle travels at the fastest speed possible, $s_{\max,\,\text{inter}}$, while the last time that the vehicle will depart from $cell_A$ is when the vehicle travels at the slowest speed possible, $s_{\min,\,\text{inter}}$. Hence, $cell_A$ must be reserved from the earliest possible arrival time until the last possible departure time. Therefore, the simulation to generate the reservation request message is done twice. In the first simulation, the vehicle moves at $s_{\max,\,\text{inter}}$ to get the earliest possible arrival time to the cells while in the second simulation, the vehicle moves at $s_{\min,\,\text{inter}}$ to get the last possible departure time from the cells.

We discretize the simulation time using time interval $\Delta t'$. We get the change in the position of the vehicle at every time interval of $\Delta t'$; thus, smaller $\Delta t'$ will give more accurate movement of the vehicle in the simulation. Also, a vehicle is to reserve a cell for at least one $\Delta t'$; thus, the arrival time and departure time in each cell are in the unit of $\Delta t'$. In addition, to make sure that all cells that the vehicle will pass through are found in the simulation when it travels at $s_{\max,\,\text{inter}}$, we assign an upper bound constraint to $\Delta t'$, that is

$$\Delta t' < \frac{\text{cell size} + \text{car length}}{s_{\max,\,\text{inter}}} \tag{1}$$

Basically, at every time interval $\Delta t'$, we obtain the following:

- *Vehicle's position and orientation*: the vehicle's position and orientation at a time instant is determined based on its speed, size, entrance lane, and trajectory in the intersection.
- *Cells that a vehicle occupies*: after capturing the position and orientation of the vehicle, we get the cells it will occupy in this position. We can easily check whether that vehicle occupies a cell or not by checking whether the vehicle's perimeter intersects with any of the cell borders. Note that some of the cells that are inside the vehicle's perimeters will not intersect with the cell's border, but this will not cause problem since no other vehicle can reserve those cells since the surrounding cells are to be reserved by this vehicle anyway.

Steps for the simulation are as follows:

1. Initialize *rc, rta*, and *rtd* as empty sets.
2. Execute the following steps two times: once using $s_{\max,\,\text{inter}}$ (first simulation) and once using $s_{\min,\,\text{inter}}$ (second simulation).

3. Compute d_{int} which is the distance to be traveled by the vehicle every time step $\Delta t'$ at the intersection

$$d_{\text{int}} = \begin{cases} s_{\max,\text{inter}}\Delta t', \text{1st simulation} \\ s_{\min,\text{inter}}\Delta t', \text{2nd simulation} \end{cases} \quad (2)$$

4. Start the simulation by positioning the vehicle right before the entrance of the intersection. Starting from this position, we simulate the vehicle's movement and get the cell it occupies at each time step until it exits the intersection.
   (a) Initialize $t_s = 0$, where t_s is the simulation time.
   (b) Get the vehicle's new position after traveling a distance of d_{int} from its current position.
   (c) Get the cells that the vehicle occupies on this new position.
   (d) For each cell that the vehicle occupies at the new position.
      (i) If the cell is not yet in *rc*, then insert the cell to *rc*, insert $t_s - \Delta t'$ to *rta*, and insert $t_s + \Delta t'$ to *rtd*. We subtract $\Delta t'$ from the time inserted to *rta* and add $\Delta t'$ to the time inserted to *rtd* to give allowance since we estimate the position of the vehicle by discretizing the time.
      (ii) If the cell is already in *rc*, meaning the cell was already occupied in the previous time step, then change the respective cell's *rtd* to the current value of $t_s + \Delta t'$.
   (e) Increment t_s by $\Delta t'$ for the next time step.
   (f) Repeat steps 4b to 4e until the vehicle is completely outside the intersection.

After the simulations, *rc* will contain the cells that the vehicle will request to reserve in the intersection, while *rta* will contain the arrival times to each cell in *rc* when the vehicle travels at $s_{\max,\text{inter}}$ and *rtd* will contain the departure times from each cell in *rc* when the vehicle travels at $s_{\min,\text{inter}}$. Therefore, the values in *rta* will be earliest possible arrival times to each cell in *rc*, and *rtd* will be the last possible departure times from the cells in *rc*. Hence, as long as the vehicle travels in the intersection at a speed within the speed range, the actual time that it will occupy the cells in the intersection will be within the requested reservation time.

Note that we started the simulation with the vehicle positioned right before it enters the intersection. Thus, we need to update the values of *rta* and *rtd* before the request is sent to the intersection manager so that the requested time to the cells is in accordance with the time that the vehicle can actually reach the intersection from its current position when it sends the request. Thus, before it sends the request, the values of *rta* and *rtd* must be updated by adding the earliest time the vehicle can reach the entrance of the intersection from its current position, to every value in *rta*, and adding the last time the vehicle can reach the entrance of the intersection from its current position, to every value in *rtd*. Finally, after updating the request, the values of *rc*, *rta*, and *rtd* are the information to be sent to the intersection manager for the reservation request.

Intersection manager schedules the requests

As previously mentioned, the intersection manager processes the requests at every Δt, the time step size. At time instant t_i, the intersection manager processes the requests received during $[t_{i-1}, t_i)$. Hence, there can be more than one reservation requests to process.

Specifically, the intersection manager does the following at every scheduling time instant t_i:

1. Determine the best sequence of the vehicles' entrances to the intersection.
2. Make reservations based on the chosen sequence and send response to the vehicles.
3. Update the times of current reservations for the next scheduling time.

We discuss each task in detail in the following sections.

Determining the best sequence of vehicles to be scheduled. Since there can be many vehicles to be scheduled at every scheduling phase, we determine the best sequence of vehicles' entrances to the intersection that will best utilize the intersection. Genetic algorithm is used as the optimization technique to find the best sequence of vehicles. Since it is impossible to predict arrivals of all vehicles, we find an optimal sequence for the vehicles to be scheduled at t_i.

In the following sections, which vehicles are to be involved at each scheduling phase is first discussed in the "Vehicles involved in scheduling" section. How to schedule a vehicle is discussed in the "Scheduling a vehicle" section, and the complexity of the scheduling is discussed in the "Complexity of the scheduling algorithm" section. The application of genetic algorithm to our scheduling to be able to determine the best sequence of vehicles is explained in the "Using genetic algorithm to determine the best sequence of vehicles to be scheduled" section.

Vehicles involved in scheduling. The vehicles that are to be scheduled at t_i are categorized as follows:

- Vehicles that arrived in the queuing zone from $[t_{i-1}, t_i)$. These are the vehicles that sent reservation requests for the first time.
- Vehicles that are already in the queuing zone before t_{i-1} and have already previously sent requests but with rejected reservations. These are the vehicles that were already included in the previous scheduling, but their requests got rejected so that they will be included in the current scheduling phase. When to accept and reject a request will be discussed later.

Scheduling a vehicle. To schedule a vehicle and manage the reservations, the major operations needed to be done to the data are as follows: insertion for storing the scheduled reservations, searching from the current reservations to check whether the vehicle can enter the intersection without colliding with other vehicles, searching for the best time that the vehicle can enter the intersection, and update and delete to maintain the current reservations. Hence, an appropriate data structure should be used for the benefit of algorithm's complexity. We use red–black trees to store the reservations. Red–black tree is a balanced binary search tree; thus, it performs faster searching, insertion, and deletion compared to other data structures that are not binary search trees. Also, for insertion and deletion, red–black trees perform better than AVL trees since AVL trees need to perform more rotations than red–black trees to make it balanced. In our scheme, one red–black tree is used for each cell in the intersection, and the nodes in the tree represent the vehicles that are currently reserved for that cell. A node in the tree contains the arrival and departure times of a reserved vehicle for that cell.

For each vehicle to be scheduled, the intersection manager must make sure that the requested time duration to each cell in *rc* will not conflict with the current reservations. Let rc_k be the kth cell in *rc*, where $1 \leqslant k \leqslant m$, and m is the number of cells the vehicle will occupy. Let rta_k be the requested time of arrival to rc_k and let rtd_k be requested time of departure from rc_k. For every rc_k, we access its red–black tree and search over the tree to check whether the cell is available on the requested time through the function *checkAvailability*. If the cell is available, we go to the next cell in *rc* and continue checking the availability of the remaining cells until we have checked all of the cells in *rc*. On the contrary, when rc_k is not available on the requested time interval (rta_k, rtd_k), we search for the earliest time that the cell will be available through the function *findTime*. The function *findTime* returns a number, denoted as *inc*, that will be added to rta_k and rtd_k so that the cell is available at the time interval ($rta_k + inc$, $rtd_k + inc$). However, the value of *inc* should also be added to all of the elements in *rta* and *rtd*; therefore, we must check again all of the previously checked cells in *rc* if they are still available on the new values of *rta* and *rtd*. To check again all of the previously checked cells in *rc*, we repeat *checkAvailability* again from the first cell in *rc* and if the already checked cell is now not available, then we proceed to *findTime*. Details on the functions *checkAvailability* and *findTime* are shown in the following sections of the article. After finding the available time for all of the requested cells in *rc*, the final values in *rta* and *rtd* are the times that the vehicle can reserve each cell without collision.

Algorithm 1. Scheduling a vehicle

```
FOR each k in rc
  T=rc_k's Red-Black Tree
  Traverse T and check if there is conflict: x=checkAvailability
  (rta_k, rtd_k, T)
  IF cell is available THEN move to next cell, set k=k + 1
  ELSE
    Traverse T and find the earliest time the cell is available:
    inc = findTime(x, rta_k, rtd_k, T)
    Add inc to all of elements in rta and rtd
    Set k = 0 to check if the new values of rta and rtd are
    available starting from the first cell
  END IF
End FOR
```

The pseudocode of checking the availability of the requested cells and finding the earliest available time of the cell when it is not available is shown in Algorithm 1.

checkAvailability function. In this function, we check whether the cell is available on the requested time. Basically, we search over the cell's tree to check whether there is conflict with the requested time and the current reservations. This function returns null if the cell is available and returns the node where there is conflict otherwise.

Let T be the red–black tree of the cell in rc_k. Let x be a node in T. Let x_{ta} be the arrival time of node x. Let x_{td} be the departure time of node x. Let x_l be the left node of x, and x_r be the right node of x. We go through the nodes of T until there is conflict or until the end of the tree has been reached. First, we set x to the root of T. Then, if $rtd_k \leqslant x_{ta}$, we go to the left of x; hence, $x = x_l$. Else, if $rta_k \geqslant x_{td}$, we go to the right of x; hence, $x = x_r$. Otherwise, it means there is a conflict so the cell is not available and node x will be the returned value. The pseudocode of *checkAvailability* is shown in Algorithm 2.

As an example, consider the red–black tree T of a cell as shown in Figure 3, and we consider a case where the cell is not available, for example, when $rta_k = 15$ and $rtd_k = 20$, since it will conflict with the values in node D. First, we set x to the root of T; thus, $x =$ node A. $rta_k > x_{td}$, thus $x = x_r =$ node C. Then, $rtd_k = x_{ta}$, hence $x = x_l =$ node D. Then, $rtd_k > x_{ta}$ and $rta_k < x_{td}$, which means there is a conflict between [15, 20] and

Algorithm 2. *checkAvailability* function

```
checkAvailability (rtak, rtdk, T) {
   Set cellAvailable=true
   node x=T_root
   WHILE (x is not null and cellAvailable is true)
      IF (rtd_k ⩽ x_ta) x = x_l
      ELSE IF (rta_k ⩾ x_td) x = x_r
      ELSE set cellAvailable=false
   END WHILE
   RETURN x
}
```

Algorithm 3. *findTime* function

```
findTime (x, rtak, rtdk, T) {
   int = rtd_k − rta_k
   gap = 0
   WHILE (int>gap and x is not null)
      node s=getSuccessor(T, x)
      gap = s_ta − x_td
      IF (int ⩽ gap) inc = x_td − rta_k
      ELSE x = s
   END WHILE
   RETURN inc
}
```

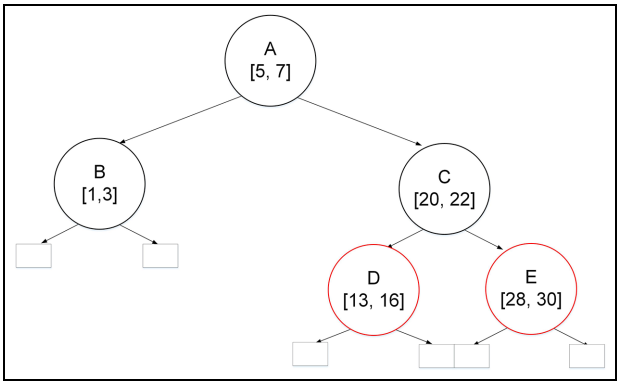

Figure 3. Example of a red–black tree of a cell.

[13, 16]; thus, the cell is not available and node D is returned wherein the conflict occurred.

findTime function. When *checkAvailability* did not return null, it means the cell in rc_k is not available at the requested time; hence, we find the earliest time that the vehicle can occupy that cell. In this function, we find the value of *inc*, which is to be added to the values of elements in *rta* and *rtd*.

Let node x be the returned node of *checkAvailability* where the conflict occurred. Let s be the successor of x which is the node whose arrival time is the smallest of

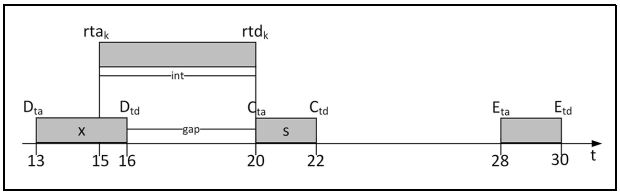

Figure 4. When x is node D and s is node C, $int > gap$.

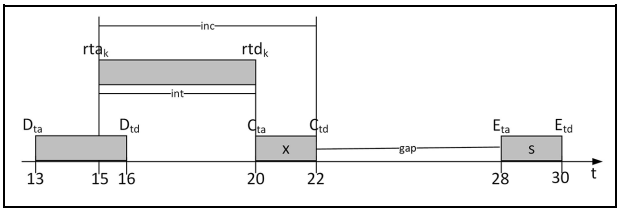

Figure 5. When x is node C and s is node E, $int < gap$; thus, we get the value of *inc*.

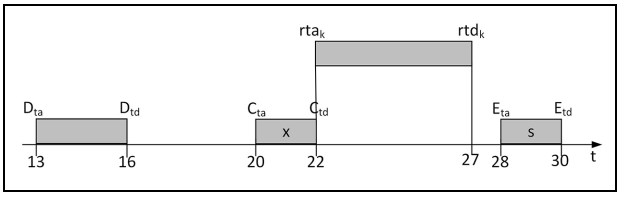

Figure 6. New values of rta_k and rtd_k after adding *inc*.

all the nodes greater than x. Let s_{ta} be the arrival time of s, and x_{td} be the departure time of x. Let $int = rtd_k - rta_k$ and $gap = s_{ta} - x_{td}$.

Starting from x, we find s such that $int \leqslant gap$ which means the time interval of the request is less than or equal to the gap between two adjacent nodes. This means we can reserve the vehicle between x_{td} and s_{ta} with new value of $rta_k = rta_k + inc$ and $rtd_k = rtd_k + inc$, where $inc = x_{td} - rta_k$. The pseudocode of *findTime* is shown in Algorithm 3.

Let us use the example considered in "checkAvailability function" section where the cell is not available when $rta_k = 15$ and $rtd_k = 20$, and red–black tree of a cell is as shown in Figure 3. As illustrated in Figure 4, set $int = rtd_k - rta_k$. From *checkAvailability*, conflict occurred at node D; thus, $x =$ node D. Then, we get the successor of x that is $s =$ node C, and we get $gap = s_{ta} - x_{td}$. As shown in Figure 4, $int > gap$; thus, we continue to get the successor of x. For the next iteration, we set $x = s =$ node C and $s =$ node E, as illustrated in Figure 5. Then, we compute $gap = s_{ta} - x_{td}$. Now, $int < gap$; thus, we can compute $inc = x_{td} - rta_k$. The new value of $rta_k = rta_k + inc$ and $rtd_k = rtd_k + inc$ which means the cell can be reserved to this vehicle the earliest without colliding with other reserved vehicles at the new time interval $[rta_k, rtd_k]$, as illustrated in Figure 6.

Complexity of the scheduling algorithm. Denote n as the maximum number of nodes of all the trees. The *checkAvailability* is a standard search operation in a red–black tree; hence, its complexity is $O(\log_2 n)$. For *findTime*, we traverse the tree until we find a slot, where we can insert the requested time interval and it does not conflict with any of the current values in the nodes. Thus, at worst case, the vehicle can be reserved only after traversing all of the nodes in the tree which makes the complexity of *findTime* $O(n)$.

When *checkAvailability* detects a conflict on rc_k, then it means *checkAvailability* was already executed k times. In addition, the function *findTime* will only be executed when there is a conflict in *checkAvailability*. Therefore, this gives a complexity of $O(k \log_2 n + n)$. The maximum number of times that the *checkAvailability* function is run for rc_k is equal to the number of vehicles already reserved to rc_k which is equal to the number of nodes in its tree. Hence, this gives the scheduling of one cell a complexity of $O(n(k \log_2 n + n))$. Since we check for all the cells in rc, the complexity becomes $\sum_{k=1}^{m} n(k \log_2 n + n)$. Hence, the overall complexity of scheduling a vehicle is $O(mn^2)$.

Using genetic algorithm to determine the best sequence of vehicles to be scheduled. To apply genetic algorithm in our scheduling, we represent the vehicle as a gene while one chromosome contains a sequence of genes that represents the order of the scheduling of the vehicles. In generating a chromosome, we have a condition that for vehicles coming from the same lane, the vehicle ahead should be scheduled first. The size of the chromosome is equal to the number of vehicles to be scheduled.

Basically, we generate multiple chromosomes and for each chromosome, we schedule the vehicles based on their order in the sequence. Different chromosomes with different scheduling sequences will give different results to the travel times of the vehicles, queue length of the lanes, throughput in the intersection, and so on. Thus, for each chromosome, we compute its fitness value based on the objective function. Then, from the current set of chromosomes, we select the chromosomes with the top fitness values and include them in the next generation. After selecting the best chromosomes, crossover and mutation are performed to generate new chromosomes for the next generation. The process of selection, crossover, and mutation are repeated to refine the population until the algorithm is terminated. After performing the genetic algorithm, we make reservations based on the sequence of vehicles of the best chromosome.

To give priority to emergency vehicles, we partition the chromosome into two groups which we name as class 1 and class 2. The vehicles in the class 1 are placed at the first part of the chromosome sequence followed by the vehicles in class 2. Class 1 contains the emergency vehicles and the ordinary vehicles that are ahead of the emergency vehicles, while class 2 contains the remaining ordinary vehicles. For each class, we generate random sequences of the vehicles but with condition that for every vehicle, the vehicle ahead(s) must appear in the sequence first. By doing this, we can make sure that the vehicles in class 1 will always be scheduled earlier than the vehicles in class 2, and at the same time, we still get the optimal scheduling for each class. To give conditional priority to other types of vehicles such as transit vehicles, another class can be added and its priority can be reflected accordingly by its assigned class order.

Making reservations based on the chosen sequence and sending response to vehicles. After finding the best sequence through genetic algorithm, we finalize the reservations based on that sequence. However, for each vehicle in the chosen chromosome, a reservation can either be accepted or rejected. If a vehicle's reservation is accepted, then the reservation is made by inserting the vehicle's requested arrival and departure times to their respective cell's red–black trees. Also, the intersection manager will send an acceptance message to the vehicle. Otherwise, it will not save the reservation and will send a rejection message to the vehicle.

When to accept or reject a reservation schedule. After the scheduling phase that started at t_i is completed, not all of the vehicles' requests can be accepted. If the vehicle is scheduled to enter the acceleration zone after t_{i+2}, then this means that the vehicle will still be in the queuing zone by the time the next scheduling phase is finished. Hence, the vehicle can still be included in the next scheduling. Therefore, a schedule is rejected if it is scheduled to enter the acceleration zone after t_{i+2}, and the vehicle will have to send the request again so that it will be scheduled again at t_{i+1}. On the contrary, a schedule is accepted if it is scheduled to enter the acceleration zone before t_{i+2}.

Update the times of current reservations for next scheduling time. Since the saved reservation times are relative to the latest scheduling time instant, the values in all of the nodes of every cell's tree should be updated by decreasing the values of the saved arrival and departure times. Also, a reservation will be deleted when the reserved time to the cell has already expired. This means that a node is deleted from the tree when the node's departure time is zero which means the vehicle has already departed the cell.

Vehicles can travel in intersection at any speed within the allowed speed range

High throughput across the intersection can be achieved by making the vehicles pass the intersection at

Table 1. Simulation parameters used for different values of speed range.

Case	$s_{\min,\text{inter}}$ (m/s)	$s_{\max,\text{inter}}$ (m/s)	l_a (m)	$a_{\min}$ (m/s^2)	$a_{\max}$ (m/s^2)
A	5.48	9.48	10	1.5	4
B	6.32	10.32	10	2	4
C	7.75	11.75	10	3	4
D	8.94	12.94	20	2	4
E	10.95	14.95	15	4	4
F	10.95	14.95	20	3	4
G	11.83	15.83	20	3.5	4
H	12.25	16.25	30	2.5	4
I	12.65	16.65	20	4	4
J	14.49	18.49	30	3.5	4

a constant and high speed. However, it is not practical in real-world system to assume that all vehicles can always travel at a constant speed in the intersection; thus, we relax this assumption and allow the vehicles to move at any speed within $[s_{\min,\text{inter}}, s_{\max,\text{inter}}]$.

We determine the values of $s_{\min,\text{inter}}$ and $s_{\max,\text{inter}}$ such that it is possible for the vehicles to reach at a speed within that range by the time they enter the intersection. They are determined as a function of the length of the acceleration zone, the acceleration rate of the vehicles, and the initial speed of the vehicles when they enter the acceleration zone.

Let l_a represent the length of the acceleration zone, and assuming that the vehicles can accelerate at a rate within $[a_{\min}, a_{\max}]$ in the acceleration zone, a vehicle accelerating at a_{min} from standstill will reach the speed of $s_{\min,\text{inter}}$ by the time it enters the intersection entrance where

$$s_{\min,\text{inter}} = \sqrt{2a_{\min}l_a} \tag{3}$$

The value of $s_{\max,\text{inter}}$ can be obtained by adding the predefined value of speed range to $s_{\min,\text{inter}}$ as follows

$$s_{\max,\text{inter}} = s_{\min,\text{inter}} + sr \tag{4}$$

where sr is the speed range but with the following constraints

$$\begin{aligned} &\sqrt{2l_a a_{\max}} - s_{\min,\text{inter}} \leqslant sr \\ &\leqslant \sqrt{s_{\max,\text{lane}}{}^2 + 2l_a a_{\max}} - s_{\min,\text{inter}} \end{aligned} \tag{5}$$

which is equivalent to

$$\sqrt{2l_a a_{\max}} \leqslant s_{\max,\text{inter}} \leqslant \sqrt{s_{\max,\text{lane}}{}^2 + 2l_a a_{\max}} \tag{6}$$

The bounds on the sr and $s_{\max,\text{inter}}$ are to guarantee that it is possible for a vehicle to reach $s_{\max,\text{inter}}$ by the time it enters the intersection. A vehicle will enter the intersection with the speed equal to the upper bound of $s_{\max,\text{inter}}$ when it enters the acceleration zone at $s_{\max,\text{lane}}$ and accelerates at $a_{\max}$. A vehicle will enter the intersection with the speed equal to the lower bound of $s_{\max,\text{inter}}$ when it enters the acceleration zone from full stop and accelerates at $a_{\max}$.

In summary, the variables $a_{\min}$, $a_{\max}$, l_a, and sr can be predetermined in our algorithm, and their values determine the values of $s_{\min,\text{inter}}$ and $s_{\max,\text{inter}}$. To consider the acceleration of the heavy and light vehicles, we set $a_{\min}$ as the minimum acceleration rate of the heavy vehicles and set $a_{\max}$ as the maximum acceleration of light vehicles.

Performance evaluation

Performance for different values of $s_{\min,\text{inter}}$ and $s_{\max,\text{inter}}$

Simulations were performed using different values of $s_{\min,\text{inter}}$ and $s_{\max,\text{inter}}$ to see the effect of their values to the system's throughput performance. The parameter values used in the simulations that we run for this section are listed in Table 1. In all of the simulations, we also use the cell size of 5 m, Δt of 5 s, $\Delta t'$ of 0.1 s, sr of 4 m, and $a_{\max}$ of 4 m/s^2, and all of the vehicles move straight. In every simulation, we increased the values of l_a and $a_{\min}$ (which in effect will also increase the values of $s_{\min,\text{inter}}$ and $s_{\max,\text{inter}}$) to see the effect of these variables to the performance. We compared the performance for different cases by their throughput. The throughput is measured by the average number of vehicles that entered the intersection within a period of time.

In contrast to our intuition that the system with highest $s_{\min,\text{inter}}$ and $s_{\max,\text{inter}}$ will show the best performance (or highest efficiency in the intersection) simply because vehicles can pass intersection quickly, it has been observed that the cases with highest value of $a_{\min}$ give the best throughput performance and the throughput decreases as $a_{\min}$ decreases, as shown in Figure 7.

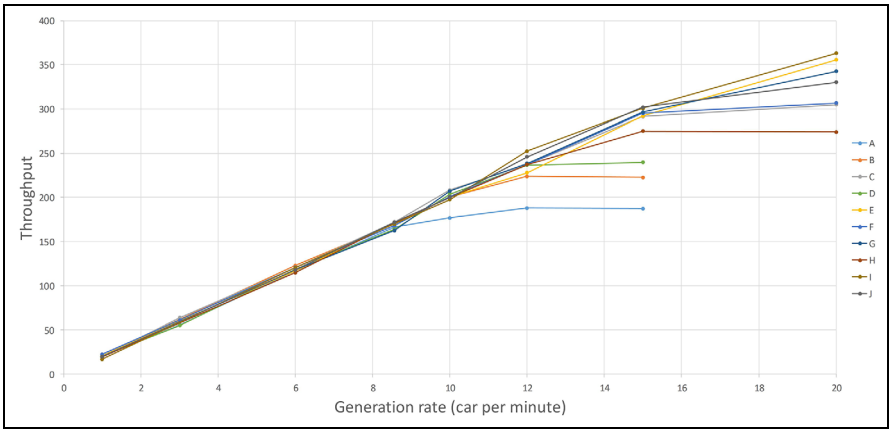

Figure 7. Comparison of the intersection's throughput for different values of $s_{min,\,inter}$, $s_{max,\,inter}$, l_a, and a_{min}.

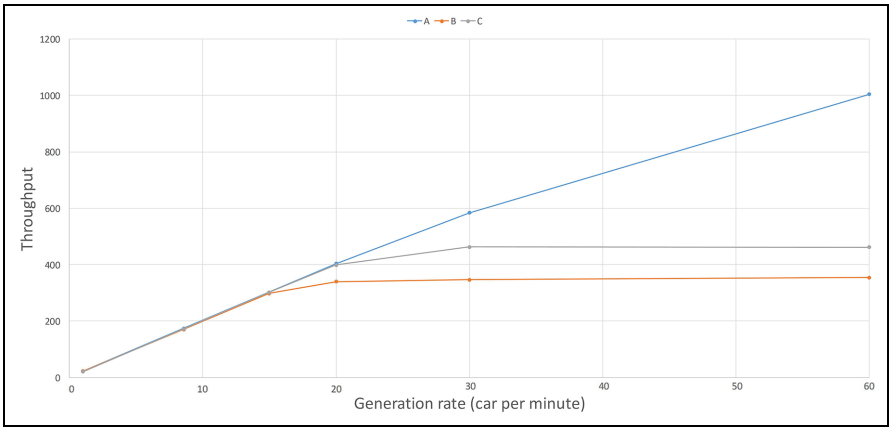

Figure 8. Comparison of the intersection's throughput when vehicles must travel at constant speed versus when the vehicles are allowed to travel at any speed within the speed range.

We have observed that increasing the value of a_{min} decreases the reserved time duration to the cells, which in effect decreases the distance between the vehicles in the intersection and thus increases the throughput.

Effect on the performance of allowing a speed range for vehicle's speed

Figure 8 shows the performance of two systems, one is allowing the vehicles to move at any speed within a speed range and another is allowing them to move only at a constant speed. The simulation parameters for the speed range are shown in Table 2. As expected, allowing the vehicles to move at any speed within a speed range decreases the intersection's throughput. This is because it is necessary to make the distance among the nearby vehicles greater in the intersection to have enough space and avoid collision in the case when any of the vehicles changes its speed. It is also observed that using faster constant speed gives higher throughput compared to slower constant speed.

Table 2. Simulation parameters used for simulation results in Figure 8.

Case	$s_{min,\,inter}$ (m/s)	$s_{max,\,inter}$ (m/s)
A	20	20
B	11.83	15.83
C	12.65	12.65

Objective functions for genetic algorithm

To improve the efficiency in the intersection, we tested different objective functions in our scheduling. In this section, we will show two objective functions that we tried in our scheduling and compare their effects to the scheduling results. We run simulations using the

Table 3. Simulation parameters used for objective functions 1 and 2.

$l_a = 20\,m$	$sr = 4\,m$	$\Delta t = 5\,s$	Crossing plan 2
$a_{min} = 3.5\,m/s^2$	$s_{min,\,inter} = 11.83\,m/s$	$\Delta t' = 0.1\,s$	Inter-arrival time per way = 1 s
$a_{max} = 4\,m/s^2$	$s_{max,\,inter} = 15.83\,m/s$	Cell size = 5 m	All vehicles are ordinary and move straight

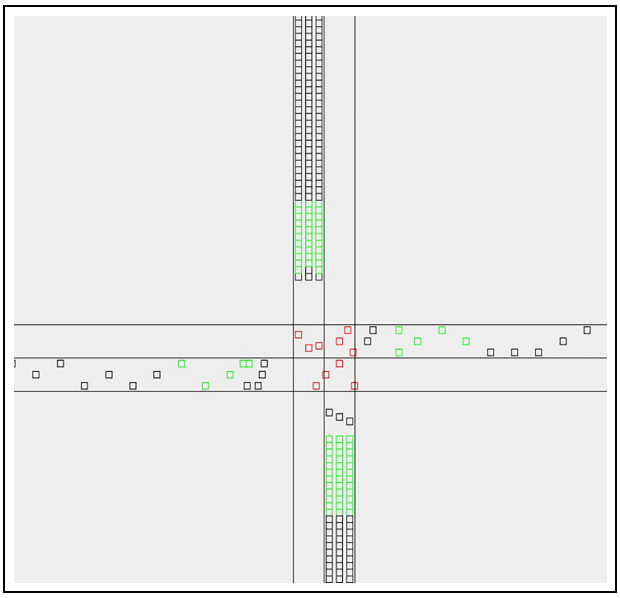

Figure 9. Screenshot of the simulation using objective function 1.

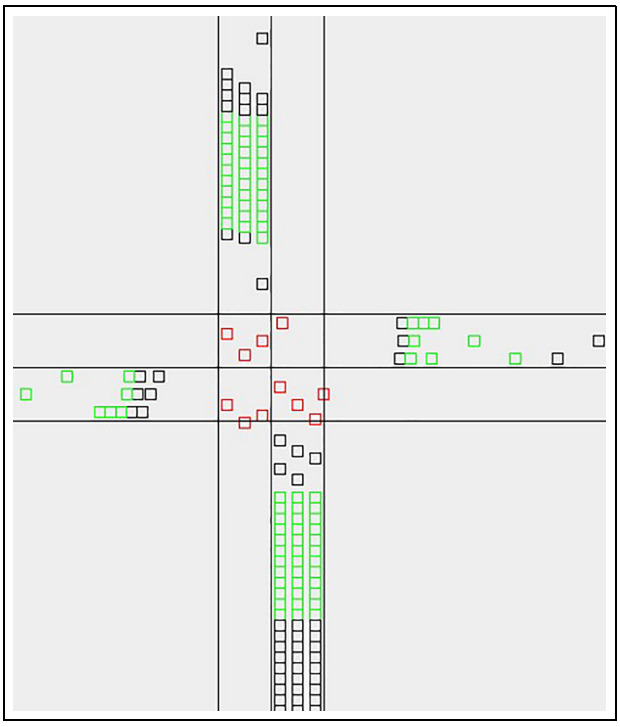

Figure 10. Screenshot of the simulation using objective function 2.

simulation parameters listed in Table 3 for the two objective functions.

Objective Function 1: minimize the total delay of vehicles. Using this objective function, the goal is to minimize the total delay of all of the vehicles that are to be scheduled at the current scheduling phase. A screenshot of the simulation using this objective function is shown in Figure 9.

As shown in the figure, the lanes in the north and south ways have longer queues which means they are serviced less frequently compared to the lanes in the east and west ways. This is because the total delay of the vehicles to be scheduled in the current scheduling phase is minimized if we let the vehicles that are on the lanes used by those vehicles in the intersection enter the intersection first. For example, if during the current scheduling phase, the vehicles that are in the intersection are from the east and west ways, then the vehicles in the east and west ways should enter the intersection first to minimize the total delay because they can enter the intersection with smaller delay than the vehicles from north and south ways. And since we allow rescheduling, if the vehicles in the north and south ways will still be in the queuing zone by the time the next scheduling phase ends, then they will be included again in the next scheduling. In the next scheduling, if the current vehicles in the intersection are still from the east and west ways, then the same decision will be made so the vehicles in the east and west ways will be scheduled to enter the intersection first. Because of this issue, we use another objective function which is discussed in the next section.

Objective Function 2: maximize the number of vehicles with accepted reservation. Another objective function we used is to maximize the number of vehicles with accepted reservations, which means we maximize the number of vehicles that will enter the intersection for the following two scheduling times, $2\Delta t$. This objective is the same as minimizing the total queue length. A screenshot of the simulation is shown in Figure 10. From this, we can see that this objective does not cause unfair scheduling among lanes.

In this simulation, the lanes are alternately serviced by group. The vehicles in the east and west ways enter

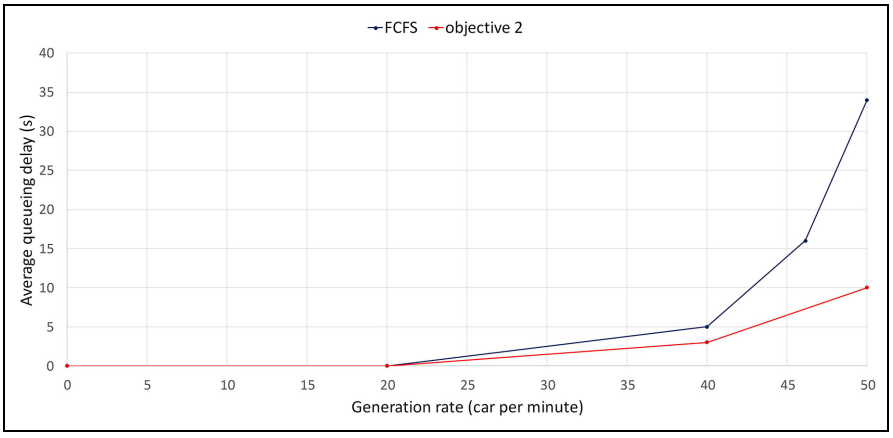

Figure 11. Comparison of queuing delays for scheduling using FCFS and genetic algorithm.

the intersection for a certain period of time, while the queues in the north and south ways are building up. Then after some time, the vehicles in north and south ways enter the intersection for a certain period of time, while the queues build up in the east and west ways. As a result, the problem experienced with objective function 1 is solved. Hence, we use objective function 2 in our scheduling algorithm. In the next section, we compare the result of FCFS with the optimization technique using objective function 2.

Comparison of FCFS scheduling with scheduling using genetic algorithm

We also compared the performance of scheduling using genetic algorithm and FCFS scheduling. We run simulations using the simulation parameters in Table 3 except for the inter-arrival time of the vehicles and the trajectory of the vehicles. In this section, we use different inter-arrival times to see the performance for different traffic rates. We compare their performance through the queuing delay, which is measured as the time spent from the time a vehicle stopped due to a queue of vehicles ahead until the time it entered the intersection.

In our simulation, 15% of vehicles turn left, 60% of vehicles move straight, and 25% of vehicles turn right per way. In Figure 11, we can see that our scheduling scheme using objective function 2 always performs better than FCFS. One problem with FCFS is that since the vehicles are scheduled individually, the vehicles from different lanes cross the intersection alternately. This causes more interruptions to the flow of traffic in the intersection and thus increasing the average queue delay and decreasing the throughput in the intersection. On the contrary, our scheduling algorithm overcomes this problem since it processes multiple vehicles in scheduling and it allows vehicles to enter the intersection by group, thereby decreasing the interruptions in the flow of traffic.

Comparison of the queuing delay experienced by emergency vehicles and ordinary vehicles in class 2

We also compared the queuing delay experienced by the emergency vehicles with the ordinary vehicles in class 2. Specifically, for each simulation, we selected one scheduling phase with one emergency vehicle to be scheduled and obtained the queuing delay of the emergency vehicle and of all the ordinary vehicles included in that scheduling phase. We define the queuing delay here as the time the vehicle spent waiting in the queue from the time the scheduling has started until the time the vehicle has entered the intersection. We run simulations using objective function 2 and the simulation parameters in Table 3 except that we included emergency vehicles, and we also run simulations on different traffic rates. We obtained the average queuing delay of the vehicles after running multiple simulations on each traffic rate.

The result of the simulations is shown in Figure 12. Based on the results, the emergency vehicles experience shorter queuing delay than the vehicles in class 2. Also, as the traffic rate increases, the queuing delay of the vehicles in class 2 increases faster than the emergency vehicles. Therefore, the result shows that we can successfully prioritize the emergency vehicles in our scheduling scheme.

Scheduling run time

To evaluate the applicability of our scheduling scheme to real-world system, we also obtained the actual run time of the scheduling. We use a processor of Intel Core i7-4790 with 3.60 GHz clock speed. The scheduling run time must be less than Δt for the schedule to be received

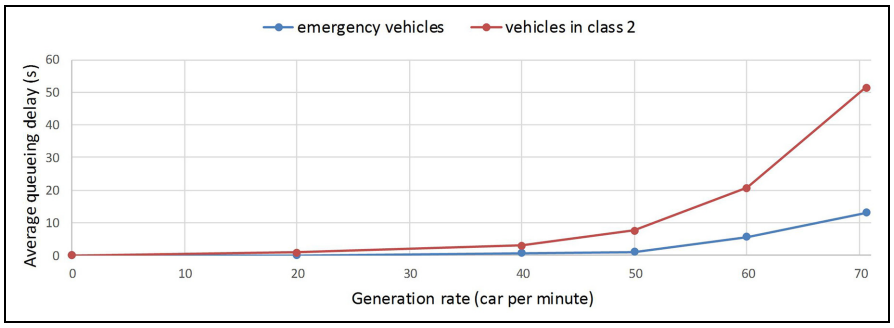

Figure 12. Comparison of queuing delays of emergency vehicles and ordinary vehicles in class 2.

by the vehicles on time. Note that scheduling run time increases as the traffic increases since more vehicles are included in the scheduling. For the highest traffic rate, where the inter-arrival time between vehicles per way is 1.2 s, which means the generation rate is around 50 vehicles per minute per way, the average run time of the scheduling is 1 s. The scheduling run time of 1 s is less than the value Δt which is 5 s. Hence, this shows that there is enough time for scheduling.

Conclusion

In this article, we presented our scheduling scheme for autonomous intersection that is based on the reservation-based scheduling approach. The intersection manager schedules the request using reservation-based approach wherein only one vehicle can occupy a space in the intersection at a time so that there is no collision in the intersection. To improve the efficiency in the intersection, genetic algorithm is used to choose the optimal sequence of vehicles to be scheduled. In the genetic algorithm, the objective used is to maximize the number of vehicles with accepted reservations at each scheduling phase. Also, red–black tree is used to store the reservations and efficiently manage the reservations. We also made the system more applicable to real-world system by relaxing the constraint on the intersection speed requirement and allowing the vehicles to move at any speed within the defined speed range.

For the performance evaluation of our work, we showed results on the effect of the speed range to the throughput in the intersection. We also showed the results on two different objective functions that we tested and showed their effects to the performance of the intersection. Finally, using the second objective function, simulation results show that our scheduling scheme performs better than the FCFS scheduling which is the approach used by other studies that use the reservation-based scheduling. We also evaluated the actual scheduling run time to show that our scheduling scheme can be applicable to real-world systems.

Declaration of conflicting interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The author(s) disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This research was supported by the Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education (NRF-2018R1D1A1B07049577 and NRF-2017R1D1A1B03035324).

ORCID iD

Areeya Rubenecia https://orcid.org/0000-0001-8385-4500

References

1. Bagloee SA, Tavana M, Asadi M, et al. Autonomous vehicles: challenges, opportunities, and future implications for transportation policies. *J Mod Transp* 2016; 24(4): 284–303.
2. Dresner K and Stone P. A multiagent approach to autonomous intersection management. *J Artif Intell Res* 2008; 31: 591–656.
3. Fagnant DJ and Kockelman K. Preparing a nation for autonomous vehicles: opportunities, barriers and policy recommendations. *Transport Res A-Pol* 2015; 77: 167–181.
4. de La Fortelle A. Analysis of reservation algorithms for cooperative planning at intersections. In: *Proceedings of the 13th international IEEE conference on intelligent transportation systems*, Madeira Island, 19–22 September 2010, pp.445–449. New York: IEEE.
5. Azimi R, Bhatia G, Rajkumar R, et al. Intersection management using vehicular networks. SAE technical paper, 2012-01-0292, 2012.
6. Yan F, Wu J and Dridi M. A scheduling model and complexity proof for autonomous vehicle sequencing problem at isolated intersections. In: *Proceedings of 2014 IEEE international conference on service operations and logistics, and informatics*, Qingdao, China, 8–11 October 2014, pp.78–83. New York: IEEE.

7. Li L and Wang FY. Cooperative driving at blind crossings using intervehicle communication. *IEEE T Veh Technol* 2006; 55(6): 1712–1724.
8. Van Middlesworth M, Dresner K and Stone P. Replacing the stop sign: unmanaged intersection control for autonomous vehicles. In: *Proceedings of the 7th international joint conference on autonomous agents and multiagent systems*, vol. 3, Estoril, 12–16 May 2008, pp.1413–1416. New York: ACM.
9. Lu Q and Kim KD. Autonomous and connected intersection crossing traffic management using discrete-time occupancies trajectory. *Appl Intell* 2019; 49(5): 1621–1635.
10. Levin MW, Boyles SD and Patel R. Paradoxes of reservation-based intersection controls in traffic networks. *Transport Res A-Pol* 2016; 90: 14–25.
11. Wu J, Abbas-Turki A, Correia A, et al. Discrete intersection signal control. In: *Proceedings of the 2007 IEEE international conference on service operations and logistics, and informatics*, Philadelphia, PA, 27–29 August 2007, pp.1–6. New York: IEEE.
12. Wu J, Abbas-Turki A and El Moudni A. Cooperative driving: an ant colony system for autonomous intersection management. *Appl Intell* 2012; 37(2): 207–222.
13. Lin P, Liu J, Jin PJ, et al. Autonomous vehicle-intersection coordination method in a connected vehicle environment. *IEEE T Intell Transp Sy* 2017; 9(4): 37–47.
14. Lee J and Park B. Development and evaluation of a cooperative vehicle intersection control algorithm under the connected vehicles environment. *IEEE T Intell Transp* 2012; 13(1): 81–90.